

LeetCode 2024: Maximize the Confusion of an Exam

Companies: Arcesium, Apple

Problem Overview:

For a True or False question exam, the teacher wants to maximize confusion by maximizing the number of consecutive questions with the same answer.

k is the maximum number of times we can change the answer from 'T' to 'F' or from 'F' to 'T'.

Output: Max number of consecutive T's or F's after performing at most *k* operations.

Core Logic:

We can make a substring of all 'T's if the count of 'F's in it is $\leq k$.

Or, we can make a substring of all 'F's if the count of 'T's in it is $\leq k$.

If either is possible, we update our maximum answer.

Approach 1: Brute Force

Complexity: Time: $O(N^2)$, Space: $O(1)$

```
for(int i = 0; i < n; i++) {  
    int T = 0;  
    int F = 0;  
    for(int j = i; j < n; j++) {  
        if(arr[j] == 'T') T++;  
        else F++;  
  
        if(T <= k || F <= k) {  
            ans = max(ans, j - i + 1);  
        }  
    }  
}
```

Approach 2: Sliding Window [Double Pass]

This approach runs a sliding window twice: once to maximize 'T's (allowing at most k 'F's) and then resetting everything to run again to maximize 'F's (allowing at most k 'T's).

```

class Solution {
public:
    int maxConsecutiveAnswers(string answerKey, int k) {
        int n = answerKey.length();
        int tcnt = 0;
        int fcnt = 0;
        int l = 0;
        int r = 0;
        int ans = 0;
        int maxans = 0;

        // First pass: maximize 'T's by changing at most k 'F's
        while(r < n) {
            if(answerKey[r] == 'T') tcnt++;
            else fcnt++;

            while(fcnt > k) {
                if(answerKey[l] == 'T') tcnt--;
                else fcnt--;
                l++;
            }
            ans = max(ans, r - l + 1);
            r++;
        }

        maxans = ans;

        // RESET EVERYTHING and run another loop (this time for tcnt > k)
        ans = 0;
        l = 0;
        r = 0;
        tcnt = 0;
        fcnt = 0;

        // Second pass: maximize 'F's by changing at most k 'T's
        while(r < n) {
            if(answerKey[r] == 'F') fcnt++;
            else tcnt++;

            while(tcnt > k) {
                if(answerKey[l] == 'F') fcnt--;
                else tcnt--;
                l++;
            }
            ans = max(ans, r - l + 1);
            r++;
        }
    }
}

```

```

        maxans = max(maxans, ans);
        return maxans;
    }
};

```

Approach 3: Sliding Window [Single Pass]

Similar to LeetCode 424 (Longest Repeating Character Replacement). We maintain the frequency of both characters. The number of changes needed in the current window is $(\text{window_size}) - \text{max_frequency}$. If this exceeds k , we shrink the window from the left.

```

class Solution {
public:
    int maxConsecutiveAnswers(string answerKey, int k) {
        int n = answerKey.size();
        vector<int> freq(2, 0);
        int l = 0;
        int r = 0;
        int maxfreq = 0;
        int ans = 0;

        while(r < n) {
            if(answerKey[r] == 'T') freq[0]++;
            else freq[1]++;

            // Max frequency of any character in the current window
            maxfreq = max(freq[0], freq[1]);

            // If elements to change exceed k, shrink window
            while((r - l + 1) - maxfreq > k) {
                if(answerKey[l] == 'T') freq[0]--;
                else freq[1]--;
                l++;
            }

            ans = max(ans, r - l + 1);
            r++;
        }
        return ans;
    }
};

```